BrandLens

Round 3: The Final Build. Team Doomsday | CodeNexus India 2026 (WeCodeCoders, Unstop) | Problem: brand governance platform (Inkloom AI)

Live product: https://brandlens-74wa.onrender.com (free hosting: the first load can take up to a minute)
Source code: https://github.com/Sachin-web-design/Brandlens
Team leader: Sachin Rawat. Document built on 2026-11-13 from the repository.

1. Final status

BrandLens is a working web product: a creator pastes text or uploads an image, DOCX or PDF, runs the review, fixes or dismisses each finding (with a reason for high and medium ones), submits it, and a reviewer or manager approves or requests changes. All three improvements promised in Round 2 are built. This document also says what is **not** done or not verified, because the review is only useful if it can be trusted.

2. Completed improvements

#	Improvement (promised in Round 2)	What was delivered in Round 3	Honest limits
1	Production-grade LLM review with retrieval and a measured benchmark	BM25 retrieval over approved facts, claims and rules (app/retrieval.py). Gemini sees only the retrieved passages and must cite one of their IDs. Strict validator (app/ai.py): invented quotes rejected, no valid retrieved source ID = 'cannot verify', expired facts cannot support a claim, unsafe rewrites dropped. 50-item benchmark with answer key and a script that reports the results (app/benchmark.py, Benchmark tab).	Retrieval is keyword based, not embeddings. The AI layer needs a Gemini key and was not measured in this build environment (no key); the numbers in section 5 are for the rule layer.
2	Image and document review	Upload PNG, JPG, WEBP, GIF, DOCX or PDF (5 MB max) (app/files.py). Text extraction feeds all text checks. Images: brand-palette match, estimated text contrast, logo detection by template matching. DOCX: missing alt text on pictures, heading structure, embedded images analysed. Sample files included.	No OCR: text inside images is not read. Contrast is estimated from the two dominant colours of flat graphics. Photos are not judged on palette or contrast. Images inside PDFs are not checked.
3	Guideline clarity insights and reviewer learning loop	Insights tab finds rules that are often dismissed or called unclear, proposes clearer wording or a lower severity, and applies a change only after a Manager approves; facts and disclosure rules are never tuned (app/tuning.py).	First version committed 10 Nov 19:13 IST, before the Round 3 window opened (see section 9). Uses seeded demo history on a fresh deploy, labelled as seeded.

3. Round 2 problems fixed

- The interface was redesigned: sidebar navigation, summary cards, clearer finding cards with severity colours, a review queue table, light and dark themes, and a layout that works on narrow screens.
- A failed, slow or sleeping server used to leave a blank screen. Every request now has a 90 second timeout and shows a plain error message; the review shows a 'waking up' note.
- The Submit button was greyed out with no explanation. It now lists exactly which findings are still open and how to resolve them, and checks the title and text before sending.
- Findings from an uploaded file are stored on the server and applied again at submission (keyed by upload ID). This is demo-level protection, not tamper-proof.
- Deprecated Pydantic call replaced; runtime data files (submissions, tuning) are git-ignored.

4. Technical details

Flow. Browser (single-page app, plain HTML/CSS/JS) to FastAPI. /api/upload extracts text and runs file checks; /api/review runs the rule checks, merges file findings and, if a key is set, the AI layer; /api/submissions re-runs the rules on the server and refuses submission while high or medium findings are open and not dismissed with a reason. Roles are checked on the server.

Part	What we used
Backend	Python, FastAPI, Uvicorn, Pydantic. app/checks.py (rules), app/retrieval.py, app/ai.py, app/files.py, app/benchmark.py, app/logic.py (workflow), app/tuning.py (learning loop), app/main.py (API)
Files and images	python-multipart (upload), python-docx, pypdf, Pillow, NumPy (FFT-based normalised cross-correlation for logo detection; WCAG contrast formula)
AI (optional)	Google Gemini over HTTPS, only when GEMINI_API_KEY is set. Output must be JSON; every item is validated; AI findings are advisory, labelled 'AI', and never block submission
Data	Brand rules and facts in data/brand.json; benchmark in data/benchmark.json; submissions in a JSON file (resets when the free host restarts)
Hosting	Render (backend and frontend together), code on GitHub

5. Benchmark results (50 sample posts, fictional brand Nourish Daily)

15 clean posts and 35 posts that each have one planted issue. Written by the team; the 27 rule-targeted posts were written to match the rules, so a high score there is **not independent evidence**. The 8 paraphrased or Hinglish posts are the honest test.

Measure (rule layer, no AI)	Result
Planted issues caught, all	27 of 35 (77.1%)
Caught, issues the rules target	100.0%
Caught, paraphrased or Hinglish issues	0.0% (all 8 missed: the known weak spot)
False alarms on clean posts (high or medium)	0 of 15 (0.0%)
Findings with a rule ID and source quote	100.0%
Findings marked 'cannot verify'	10.3%

Category	Caught
accessibility	4 of 4 (100.0%)
claims	7 of 11 (63.6%)
disclosure	2 of 2 (100.0%)
facts	8 of 10 (80.0%)
terminology	3 of 3 (100.0%)
tone	3 of 5 (60.0%)

AI layer: not yet measured. It needs a Gemini key. Run `python -m app.benchmark --ai`, then rebuild this PDF. No AI-layer accuracy is claimed in this document.

6. Testing notes

Automated: 66 unit and API tests (ai: 6, api: 6, benchmark: 4, checks: 14, files: 14, retrieval: 6, tuning: 5, validator: 11), all passing at build time. They cover the rules, the validator (invented quotes, unknown or expired source IDs, unsafe rewrites), retrieval, image, DOCX and PDF handling (including corrupt, empty, oversized and unsupported files and text that tries to give instructions), the benchmark maths with a mock model, the approval workflow, the learning loop and the full upload, review, lock, dismiss, submit flow through the API.

Screen flow: tests/ui/flow.js drives the real page in a simulated browser (jsdom) against a running server: sample load, review, locked Submit, dismiss, submit, empty-title message, image, DOCX uploads, Benchmark, Brand knowledge, Insights and Queue tabs; no script errors.

Not tested: live Gemini calls (no key or network access in the build environment), the Render deployment itself, real browsers and phones, and a full accessibility audit with a screen reader.

7. Privacy, security and responsible design

- AI only suggests. Nothing is changed, approved or published automatically; a human approves.

- Every finding names and quotes its rule or approved source; missing evidence is shown as 'cannot verify' rather than guessed.
- Uploaded files are treated as data, never instructions; size, type and page limits apply; extracted text is capped.
- AI use is disclosed in the interface. With the AI check on, content goes to Google's Gemini API, so only fictional content should be used; no key is stored in the repository.
- Demo roles are a switcher, not real authentication. Server-side role checks exist, but this is not production security.
- The interface uses native buttons and form controls and has light and dark themes; no formal accessibility audit was done.

8. Known limitations and roadmap

- Rules miss paraphrased and Hinglish wording (0 of 8 on our benchmark); the AI layer is meant for these but is advisory.
- Keyword retrieval can miss passages that share no words; embeddings (pgvector) are the next step.
- No OCR, no image checks inside PDFs, estimated contrast only; Tesseract/OpenCV and a vision model are the next step.
- Real login (JWT) and a database instead of a JSON file; video and audio review; more languages; direct publishing integrations.

9. AI assistance and commit-history disclosure

Claude (Anthropic) helped write the Round 1 plan, the Round 2 code and tests, and the Round 3 code, tests and this document, at the request of the team leader, who reviews, deploys and maintains it. The team ran the 8-person survey. At runtime the product can call Google Gemini.

A first version of Improvement 3 (app/tuning.py and its tests) was committed on **10 Nov 2026 at 19:13 IST**, before the Round 3 window opened at 23:59 IST. It is visible in the repository history and has not been altered. Improvements 1 and 2, the benchmark, the bug fixes and the new tests were written after the window opened. We leave it to the organisers to decide how the earlier commit is treated.

10. Team contributions

Each member submits an individual entry and writes in their own words only what they actually did. The leader's line below is the only one filled in here.

Member	Role (Round 1 plan)	Round 3 contribution
Sachin Rawat (Team Leader)	AI and backend lead	Directed and reviewed the Round 3 build; deployed and maintains the product. Code written with Claude (see section 9).
Suryansh Puri	Frontend and UX	[to be written by the member, only if work was done]
Snehal Jasrotia	Knowledge base and checks	[to be written by the member, only if work was done]
Khushi	Research, testing and documentation	[to be written by the member, only if work was done]